

From Machine Learning to Multi-Agent AI

An 8-week data science programme with a focus on Generative AI and agentic systems

WEEKS 1-2

Machine Learning

Foundations, supervised and unsupervised learning, ensembles, time series, tuning and MLOps

WEEKS 3-8

NLP, Generative AI & Agents

Neural networks, transformers, LLMs, RAG, fine-tuning, single-agent and multi-agent systems, capstone

Contents

1	Programme overview
2	Learning outcomes
3	Schedule at a glance
4	Phase 1: Machine Learning (Weeks 1-2)
5	Phase 2: NLP, Generative AI & Agentic AI (Weeks 3-8)
6	Capstone project
7	Assessment and certification
8	Tools and environment
9	Coverage map against the Master notes
10	Delivery guidelines for trainers

1. Programme overview

This programme takes participants from the core ideas of classical machine learning through to the design and deployment of multi-agent AI systems. It runs for eight weeks. The first two weeks establish a firm grounding in machine learning, evaluation and MLOps. The remaining six weeks concentrate on natural language processing, large language models, retrieval-augmented generation and agentic systems, which together form the main focus of the programme.

The content builds directly on the programme's Master notes, which cover the machine learning, neural network, RAG and fine-tuning material. This syllabus extends those notes with the transformer, LLM and agentic topics needed to reach multi-agent AI, and arranges everything into a week-by-week plan.

Programme at a glance

Item	Detail
Duration	8 weeks, 5 training days per week (40 sessions)
Daily format	Morning: concept session and worked examples. Afternoon: guided lab and exercises. Last 30 minutes: recap and questions.
Split	Weeks 1–2 Machine Learning (25%). Weeks 3–8 NLP, Generative AI and Agentic AI (75%).
Weekly rhythm	Monday to Thursday: new material. Friday: mini-project, quiz and review.
Final output	A deployed multi-agent application presented in Week 8
Audience	Graduates, analysts and engineers moving into data science and AI engineering roles

Prerequisites

- Working Python: functions, classes, virtual environments, and reading library documentation.
- Familiarity with pandas and NumPy for loading, cleaning and summarising data.
- Basic statistics: mean, variance, distributions, correlation and probability.
- Comfort with Git and the command line. SQL basics are helpful but not required.

Participants without these should complete a short Python and statistics refresher before Day 1.

2. Learning outcomes

By the end of the programme, participants will be able to:

- Frame a business problem as a machine learning task and follow the data science lifecycle from data collection to deployment.
- Build, evaluate and tune supervised and unsupervised models, choosing metrics that fit the problem, including under class imbalance.
- Recognise and prevent overfitting, underfitting and data leakage using pipelines and proper validation.
- Track, package and serve models with MLflow, FastAPI and Docker.
- Explain how neural networks, embeddings and transformers work, and use pretrained language models through Hugging Face and LLM APIs.
- Design, build and evaluate a production-quality RAG system with hybrid retrieval, re-ranking and citation.
- Decide between prompting, RAG and fine-tuning, and apply parameter-efficient fine-tuning (LoRA and QLoRA).
- Build tool-using agents with memory, planning and human-in-the-loop controls.
- Design, orchestrate, evaluate and deploy multi-agent systems with appropriate guardrails and observability.

3. Schedule at a glance

Week	Theme	Core topics	Friday deliverable
1	ML foundations and supervised learning	Data science lifecycle, EDA, feature engineering, regression, classification, metrics, cross-validation, class imbalance	Classification project with a full evaluation report
2	Unsupervised learning, ensembles, time series and MLOps	Clustering, dimensionality reduction, bagging and boosting, ARIMA family, tuning, leakage, MLflow, FastAPI, Docker	Tuned model tracked in MLflow and served in a container
3	Deep learning and NLP foundations	Neural networks, backpropagation, optimisers, text preprocessing, TF-IDF, word embeddings, RNNs and LSTMs	Text classifier: classical baseline vs neural model
4	Transformers and large language models	Attention, transformer architecture, BERT and GPT families, tokenisation, decoding, prompt engineering, structured output	LLM-powered extraction and summarisation tool
5	Embeddings and retrieval-augmented generation	Chunking, vector databases, dense, sparse and hybrid retrieval, query rewriting, HyDE, re-ranking, RAGAS evaluation	Evaluated RAG assistant over a document set
6	Fine-tuning, alignment, evaluation and guardrails	Transfer learning, LoRA and QLoRA, instruction tuning, RLHF and DPO, LLM-as-judge, safety and prompt injection	LoRA fine-tune compared with a prompting and RAG baseline
7	Single-agent systems	Agent loop and ReAct, tool calling, memory, planning, LangGraph, Model Context Protocol, human-in-the-loop	Research agent with tools, memory and approval steps
8	Multi-agent systems and capstone	Orchestration patterns, hand-offs, shared state, agent evaluation, observability, cost control, deployment	Capstone: deployed multi-agent application and presentation

Time allocation

Phase	Weeks	Sessions	Share of programme
Machine Learning	1–2	10	25%
NLP and Generative AI (Weeks 3–6)	3–6	20	50%
Agentic and Multi-Agent AI (Weeks 7–8)	7–8	10	25%

Generative AI and agentic content accounts for three quarters of the programme. Agentic ideas such as tool calling and structured output are introduced from Week 4 so that Weeks 7 and 8 can focus on building systems.

4. Phase 1: Machine Learning (Weeks 1–2)

Phase 1 gives participants a working command of classical machine learning. It moves quickly, because it is the foundation rather than the focus, but it covers evaluation, leakage and deployment carefully since these habits carry directly into the GenAI and agentic work that follows.

PHASE 1 · MACHINE LEARNING

Week 1 · ML Foundations and Supervised Learning

Establish the data science workflow and the core supervised learning toolkit, with a strong emphasis on choosing and interpreting the right evaluation metric.

Learning objectives

- Describe the data science and software development lifecycles and where each activity fits.
- Distinguish supervised, unsupervised, semi-supervised and reinforcement learning, and ML from deep learning.
- Train and compare regression and classification models in scikit-learn.
- Evaluate models with appropriate metrics, confusion matrices, ROC-AUC and k-fold cross-validation.
- Handle class imbalance with resampling, class weights and threshold tuning without leaking data.

Daily plan

Day	Topics	Hands-on lab
1	Data science lifecycle and SDLC; ML vs deep learning; types of machine learning; train, validation and test splits	Environment setup; end-to-end walkthrough on a small tabular dataset
2	EDA and feature engineering: missing values, outliers, encoding, scaling; which models need scaled features	EDA notebook and reusable preprocessing functions
3	Regression: linear, polynomial, ridge (L2), lasso (L1), decision trees, random forest, SVR. Metrics: MAE, MSE, RMSE, R^2 , adjusted R^2 , MAPE	House-price style regression; compare models and metrics
4	Classification: logistic regression, SVM and kernels, naive Bayes, KNN, trees, random forest, XGBoost; OvR and OvO; macro vs weighted averages	Multi-model classification benchmark with k-fold cross-validation
5	Confusion matrix, precision, recall, F1, thresholding, ROC-AUC; class imbalance with SMOTE, ADASYN, undersampling and imblearn pipelines	Mini-project and Week 1 quiz
Week deliverable	An imbalanced classification project (for example fraud or churn) with a short report justifying the chosen metric, threshold and resampling strategy.	
Reference	Master notes: Data Science; Supervised Learning; Confusion Matrices, Evaluation Metrics and Class Imbalances	

PHASE 1 · MACHINE LEARNING

Week 2 · Unsupervised Learning, Ensembles, Time Series and MLOps

Complete the classical ML toolkit and learn to take a model from notebook to a tracked, containerised service.

Learning objectives

- Apply k-means, hierarchical, DBSCAN and Gaussian mixture clustering and judge cluster quality.
- Use PCA, t-SNE, UMAP and LDA for the right purposes, knowing which are for visualisation only.
- Explain bagging, boosting and stacking and when each is preferred.
- Build ARIMA-family forecasts with chronological validation.
- Diagnose bias and variance, prevent leakage with pipelines, and tune with grid, random and halving search.
- Track experiments in MLflow and serve a model with FastAPI inside Docker.

Daily plan

Day	Topics	Hands-on lab
6	Clustering: k-means, elbow method, silhouette score, WCSS, hierarchical, DBSCAN, GMM. Dimensionality reduction: PCA, t-SNE, UMAP, LDA	Customer segmentation with PCA preprocessing
7	Ensembles: bagging, random forest, boosting (AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost), stacking. Time series: components, stationarity, differencing, ACF and PACF, AR, MA, ARIMA, SARIMA, SARIMAX	Boosting vs bagging comparison; sales forecast with SARIMA
8	Overfitting and underfitting, bias and variance, data leakage, fit-on-train pipelines, hyperparameter tuning; model, data and concept drift	Refactor a leaky notebook into a clean pipeline; tuning exercise
9	MLOps: MLflow tracking and model comparison, FastAPI endpoints and status codes, Uvicorn, Streamlit, Docker and Docker Compose; data storage types; testing and Agile	Serve the Week 1 model as an API with a Streamlit front end
10	Phase 1 review and consolidation; bridge to deep learning	Mini-project, Phase 1 assessment
Week deliverable	A tuned model tracked in MLflow, served through FastAPI in a Docker container, with a champion vs challenger comparison and basic unit tests.	
Reference	Master notes: Unsupervised Learning; Time Series; Ensemble Methods; Overfitting, Underfitting, Data Leakage and Tuning; MLOps; Data Storage Types; Testing and the Agile Approach	

5. Phase 2: NLP, Generative AI & Agentic AI (Weeks 3–8)

Phase 2 is the core of the programme. It starts with how neural networks and language representations work, moves through transformers, LLMs, RAG and fine-tuning, and finishes with single-agent and multi-agent systems. Each week produces a working component, and many of these components are reused in the capstone.

PHASE 2 · NLP, GENERATIVE AI & AGENTS

Week 3 · Deep Learning and NLP Foundations

Understand how neural networks learn and how text is turned into something a model can use, laying the ground for transformers.

Learning objectives

- Explain nodes, layers, activation functions, loss, gradient descent and backpropagation.
- Use optimisers (SGD with momentum, RMSprop, Adam), dropout and early stopping in PyTorch or Keras.
- Explain vanishing and exploding gradients and the design choices that mitigate them.
- Preprocess text and build bag-of-words and TF-IDF representations.
- Explain word embeddings and sequence models, and why they led to attention.

Daily plan

Day	Topics	Hands-on lab
11	Artificial neural networks: layers, summation and activation (sigmoid, tanh, ReLU, leaky ReLU, softmax, linear); loss functions	Build a small network from scratch in NumPy
12	Training: gradient descent, backpropagation, learning rate, local minima and saddle points, optimisers, dropout, vanishing gradients. CNNs in brief	Train a network in PyTorch; experiment with optimisers and dropout
13	NLP foundations: tokenisation, normalisation, stop words, stemming and lemmatisation, n-grams, bag-of-words, TF-IDF	Sentiment classifier with TF-IDF and naive Bayes or logistic regression
14	Word embeddings: Word2Vec, GloVe, FastText; semantic similarity. Sequence models: RNN, LSTM, GRU and their limits	Explore embedding spaces; LSTM text classifier
15	From sequence models to attention; recap and review	Mini-project and Week 3 quiz
Week deliverable	A text classification comparison: a classical TF-IDF baseline against a neural model, with analysis of errors and trade-offs.	
Reference	Master notes: Neural Networks; supplementary NLP material	

PHASE 2 · NLP, GENERATIVE AI & AGENTS

Week 4 · Transformers and Large Language Models

Understand the transformer and learn to use LLMs effectively and reliably through prompting and APIs.

Learning objectives

- Explain self-attention, multi-head attention, positional encoding and the transformer block.
- Compare encoder, decoder and encoder-decoder models and their typical tasks.
- Use Hugging Face pipelines and tokenisers, and call LLMs through provider APIs.
- Control generation with temperature, top-k, top-p and maximum tokens, and manage context windows and cost.
- Write robust prompts and request structured JSON output validated against a schema.

Daily plan

Day	Topics	Hands-on lab
16	Attention and the transformer: queries, keys and values, multi-head attention, positional encoding, residuals and layer norm	Implement scaled dot-product attention
17	Model families: BERT (encoder), GPT (decoder), T5 (encoder-decoder); pretraining objectives; subword tokenisation (BPE, WordPiece)	Hugging Face pipelines for classification, NER and summarisation
18	Working with LLMs: APIs, chat message roles, system prompts, decoding parameters, context windows, tokens and cost, hallucination	Build a chat client and compare decoding settings
19	Prompt engineering: identity, instructions, examples and context; zero-shot and few-shot; chain-of-thought; prompt templates; structured output with Pydantic; introduction to function calling	Structured information extraction from unstructured documents
20	Review; limitations of LLMs and the case for retrieval	Mini-project and Week 4 quiz
Week deliverable	An LLM tool that extracts structured fields and summaries from a document set, with schema validation and a small evaluation set.	
Reference	Master notes: RAG (prompt format); supplementary transformer and LLM material	

PHASE 2 · NLP, GENERATIVE AI & AGENTS

Week 5 · Embeddings and Retrieval-Augmented Generation

Design, build and evaluate a RAG system that grounds LLM answers in private, current documents with citations.

Learning objectives

- Explain the two phases of RAG: offline indexing and online answering.
- Choose chunking strategies, chunk sizes, overlap and metadata for a given corpus.
- Select embedding models and vector stores and understand flat, HNSW and IVF indexes.
- Implement dense, sparse and hybrid retrieval, query rewriting, HyDE and re-ranking.
- Evaluate retrieval and generation separately with precision, recall, MRR, NDCG and RAGAS.

Daily plan

Day	Topics	Hands-on lab
21	Why RAG; the full pipeline; loading documents; chunking strategies (fixed, recursive, semantic, structure-aware, hierarchical); overlap; metadata	Load and chunk a document corpus with LangChain
22	Embeddings (Sentence Transformers, API models); vector databases (Chroma, pgvector, Qdrant, Pinecone); flat, HNSW and IVF indexes; metadata filtering	Build and query a vector index
23	Retrieval: cosine similarity, dense vs sparse (BM25, TF-IDF), hybrid with reciprocal rank fusion; query expansion and rewriting; HyDE; cross-encoder re-ranking	Add hybrid retrieval and re-ranking; measure the effect
24	Generation: prompt assembly with guardrails, evidence and question sections; citations; handling insufficient evidence. Evaluation: retrieval metrics, faithfulness, relevance, RAGAS, LLM-as-judge calibration against human labels	Build an evaluation set and score the pipeline with RAGAS
25	Advanced patterns overview: conversational RAG, agentic retrieval, knowledge graphs	Mini-project and Week 5 quiz
Week deliverable	A RAG assistant over a real document set, returning cited answers, with a written evaluation of retrieval and generation quality before and after improvements.	
Reference	Master notes: RAG (Retrieval Augmented Generation)	

PHASE 2 · NLP, GENERATIVE AI & AGENTS

Week 6 · Fine-Tuning, Alignment, Evaluation and Guardrails

Know when and how to adapt a model, and how to evaluate and secure LLM applications before they reach users.

Learning objectives

- Distinguish feature extraction from fine-tuning and choose between prompting, RAG and fine-tuning.
- Apply LoRA and QLoRA with Hugging Face PEFT and manage adapters.
- Explain how base models become chat models through instruction tuning, RLHF and DPO.
- Build an evaluation harness for LLM outputs, including LLM-as-judge with calibration.
- Apply guardrails against hallucination, harmful output, PII leakage and prompt injection.

Daily plan

Day	Topics	Hands-on lab
26	Transfer learning; feature extraction vs fine-tuning; the decision order: prompting, then RAG, then fine-tuning; data quality for fine-tuning	Fine-tune a small encoder model for classification
27	Parameter-efficient fine-tuning: LoRA, QLoRA, adapters and prompt tuning; merging vs separate adapters	LoRA fine-tune of a small open model
28	Alignment: instruction tuning, reward models, RLHF, DPO; evaluating fine-tuned models on base capability and target task	Compare fine-tuned model against prompting and RAG baselines
29	LLM evaluation and safety: golden datasets, LLM-as-judge, regression testing for prompts; guardrails, input and output validation, prompt injection, PII handling, responsible AI	Add an evaluation harness and guardrail layer to the Week 5 RAG app
30	Phase 2 GenAI review; introduction to agents	Mini-project and Week 6 quiz
Week deliverable	A LoRA fine-tuned model with a written comparison against prompting and RAG on the same task, covering quality, cost and risk.	
Reference	Master notes: Fine-Tuning and Transfer Learning; RAG evaluation	

PHASE 2 · NLP, GENERATIVE AI & AGENTS

Week 7 · Single-Agent Systems

Move from answering questions to taking actions: build agents that reason, use tools, remember and ask for approval.

Learning objectives

- Explain what makes a system agentic and when a workflow is better than an agent.
- Implement the reason-act-observe loop with tool and function calling.
- Add short-term and long-term memory, including retrieval-based memory.
- Build stateful agents as graphs in LangGraph with checkpoints and human-in-the-loop steps.
- Connect agents to external tools through the Model Context Protocol (MCP).

Daily plan

Day	Topics	Hands-on lab
31	Agents vs workflows; the agent loop; ReAct; tool and function calling with JSON schemas; common failure modes (loops, wrong tools, hallucinated arguments)	Build an agent loop from scratch with two tools
32	Tool design: clear descriptions, input validation, error messages the model can use; web search, code execution and database tools; RAG as a tool	Agent with search, calculator and RAG tools
33	Memory: conversation buffers, summaries, vector memory and user profiles; planning: plan-and-execute, reflection and self-critique	Add memory and a planning step
34	LangGraph: state, nodes, edges, conditional routing, checkpoints, interrupts for human approval; Model Context Protocol servers and clients	Rebuild the agent as a LangGraph graph with an approval step
35	Agent evaluation basics: task success, tool-call accuracy, trajectory review; capstone team formation and design	Mini-project, Week 7 quiz, capstone proposal
Week deliverable	A research or operations agent with at least three tools, persistent memory and a human approval gate for side-effecting actions.	
Reference	Supplementary agentic material; builds on RAG and prompting from Weeks 4–5	

PHASE 2 · NLP, GENERATIVE AI & AGENTS

Week 8 · Multi-Agent Systems and Capstone

Design, orchestrate, evaluate and deploy systems in which several specialised agents collaborate on a task.

Learning objectives

- Choose among multi-agent patterns: supervisor, sequential pipeline, hierarchical teams, hand-offs and debate or critique.
- Manage shared state, message passing and context between agents.
- Compare frameworks such as LangGraph, CrewAI and AutoGen, and know when a single agent is enough.
- Instrument agents with tracing and evaluate them on cost, latency and quality.
- Deploy a multi-agent application behind an API with guardrails and monitoring.

Daily plan

Day	Topics	Hands-on lab
36	Why multiple agents; orchestration patterns: supervisor and workers, sequential, hierarchical, hand-offs, critique and debate; agent-to-agent communication	Supervisor with two specialist agents in LangGraph
37	Shared state and context engineering; framework comparison (LangGraph, CrewAI, AutoGen); reliability, retries and fallbacks	Implement one pattern in two frameworks and compare
38	Observability and evaluation: tracing (LangSmith or Langfuse), trajectory and end-to-end evaluation, cost and latency budgets; security for agents: least privilege, sandboxing, injection through tool outputs	Add tracing, an evaluation suite and guardrails to the capstone
39	Deployment: FastAPI service, Docker Compose, configuration and secrets, monitoring and feedback loops	Capstone build day with trainer support
40	Capstone presentations, peer review and programme close	Final presentation and demo
Week deliverable	The capstone: a deployed multi-agent application with documentation, an evaluation report and a live demonstration (see Section 6).	
Reference	Supplementary agentic material; draws on MLOps (Week 2), RAG (Week 5) and evaluation (Week 6)	

6. Capstone project

Working in teams of three or four, participants build a multi-agent application that solves a realistic business problem. The proposal is agreed on Day 35 and the system is presented on Day 40. Teams are encouraged to reuse their own components from earlier weeks.

Minimum requirements

- At least two specialised agents coordinated by a clear orchestration pattern.
- At least one retrieval component (RAG) and at least two external tools.
- Structured outputs validated against schemas, and guardrails on inputs and outputs.
- A human approval step before any action with real-world side effects.
- Tracing plus an evaluation set of at least 20 test tasks with reported results.
- Deployment as a FastAPI service in Docker with a simple front end (for example Streamlit).
- A README covering architecture, setup, design decisions, limitations and cost estimates.

Example project themes

Theme	Outline
Research analyst team	A planner, a researcher using search and RAG, and a writer that drafts a cited report, reviewed by a critic agent.
Customer support desk	A triage agent routes queries to billing, technical and policy specialists, with escalation to a human.
Data analysis assistant	Agents that load data, run analysis code in a sandbox, build charts and explain results in plain language.
Recruitment screening aid	Agents that parse CVs, match against job specifications and draft interview questions, with bias checks.

Capstone marking rubric

Criterion	Weight	What good looks like
Problem framing and design	15%	Clear use case; justified choice of agents and pattern over simpler alternatives
Technical implementation	30%	Working, readable code; sensible tools, memory, state and error handling
Evaluation and quality	20%	Meaningful test set, honest metrics, evidence of iteration
Safety and reliability	15%	Guardrails, approval gates, least-privilege tools, handling of failure cases
Deployment and documentation	10%	Runs from the README; containerised; architecture explained
Presentation and demo	10%	Clear explanation of trade-offs; confident live demo; good answers to questions

7. Assessment and certification

Component	Weight	Details
Weekly quizzes	15%	Seven short quizzes on Fridays of Weeks 1 to 7, covering concepts and interpretation
Lab completion	20%	Daily labs submitted to the course repository and checked by trainers
Weekly mini-projects	30%	Friday deliverables for Weeks 1 to 7, marked against a short rubric
Capstone project	35%	Team project assessed with the rubric in Section 6, with an individual contribution component

- Completion requires an overall score of 60% or above and at least 80% attendance.
- A score of 80% or above earns a certificate with distinction.
- Participants below the threshold in any week receive a catch-up plan agreed with their trainer.

8. Tools and environment

Area	Tools
Core	Python 3.11+, Jupyter or VS Code, Git and GitHub, virtual environments
Data and ML	pandas, NumPy, scikit-learn, imbalanced-learn, XGBoost, LightGBM, statsmodels, Matplotlib, Seaborn
Deep learning and NLP	PyTorch (or Keras), Hugging Face Transformers, Datasets, Tokenizers, PEFT, NLTK or spaCy
LLMs	At least one commercial LLM API and one open model run locally or hosted
RAG	LangChain or LlamaIndex, Sentence Transformers, Chroma or pgvector, BM25, cross-encoder re-rankers, RAGAS
Agents	LangGraph, CrewAI or AutoGen for comparison, Model Context Protocol, Pydantic
MLOps and serving	MLflow, FastAPI, Uvicorn, Streamlit, Docker, Docker Compose
Observability	LangSmith or Langfuse for tracing and evaluation
Hardware	Laptop with 16 GB RAM recommended; a GPU environment (cloud notebook or shared server) for Weeks 3 and 6

Library versions change quickly in the GenAI space. Trainers should pin versions in a shared requirements file before the programme starts and check the official documentation for each framework.

9. Coverage map against the Master notes

Every section of the Master notes is placed in the programme as follows. Topics beyond the notes are listed as supplementary and require additional trainer material.

Master notes section	Week	Day(s)
Data Science (lifecycles, types of ML)	1	1
Supervised Learning	1	2–4
Confusion Matrices, Evaluation Metrics and Class Imbalances	1	5
Unsupervised Learning	2	6
Time Series	2	7
Ensemble Methods	2	7
Overfitting, Underfitting, Data Leakage and Tuning	2	8
MLOps	2	9
Data Storage Types	2	9
Testing and the Agile Approach	2	9
Neural Networks (including vanishing gradients)	3	11–12
RAG (Retrieval Augmented Generation)	5	21–25
Fine-Tuning and Transfer Learning	6	26–28
Extra notes: drift, ROC, embeddings, guardrails, LLM evaluation	1, 2, 5, 6	5, 8, 22, 29
Supplementary: NLP foundations, transformers, LLMs, prompting	3–4	13–19
Supplementary: single-agent and multi-agent systems	7–8	31–40

10. Delivery guidelines for trainers

- Keep theory to the morning and protect afternoon lab time. Participants should write code every day.
- Start each day with a ten-minute recap of the previous day and end with three key takeaways.
- Reuse one or two datasets and document sets across weeks so participants can see a system grow.
- In Phase 2, show the simplest working solution first (a prompt, then RAG, then an agent) before adding complexity.
- Treat evaluation as part of every build, not an afterthought. Each mini-project must report a measurable result.
- Model good engineering habits throughout: version control, pinned dependencies, tests and clear READMEs.
- Flag fast-moving areas, especially agent frameworks, and refresh examples before each cohort.

Suggested daily timetable

Time	Activity
09:30 – 10:00	Recap and questions from the previous day
10:00 – 12:30	Concept session with worked examples and short exercises
12:30 – 13:30	Lunch
13:30 – 16:30	Guided lab (Fridays: mini-project work and quiz)
16:30 – 17:00	Review of lab outcomes and key takeaways

Weekly checkpoints

Checkpoint	When	Purpose
Quiz	Friday of Weeks 1–7	Confirm understanding of the week's concepts
Mini-project review	Friday of Weeks 1–7	Assess applied skills and give written feedback
Phase 1 assessment	Day 10	Gate before the deep learning and GenAI phase
Capstone proposal	Day 35	Agree scope, architecture and team roles
Capstone presentation	Day 40	Final assessment and programme close